

Informe de Sistema Distribuido - File Search

2da Entrega - Sistemas Distribuidos 2025

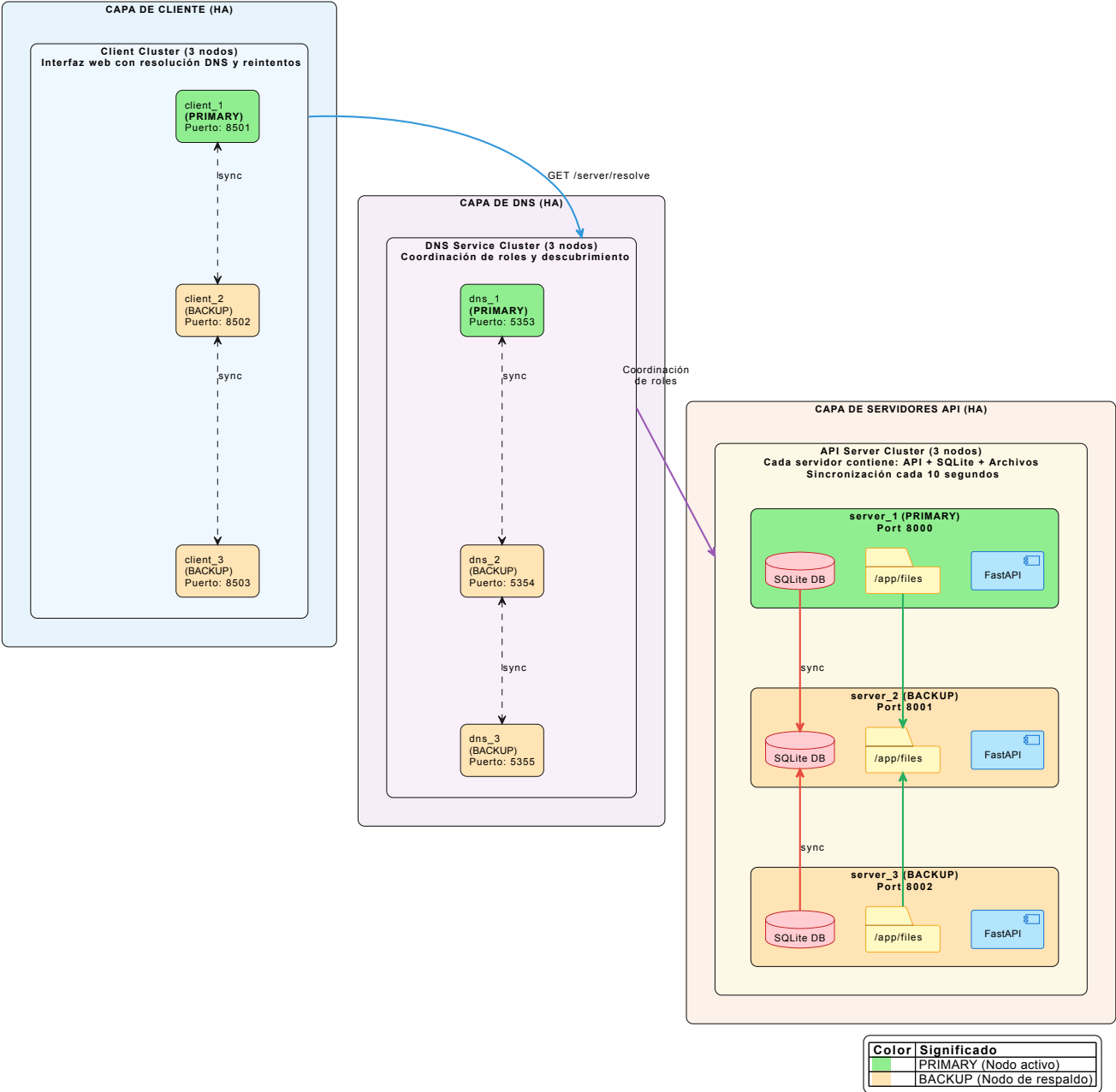
Índice

1. [Arquitectura](#)
 2. [Procesos](#)
 3. [Comunicación](#)
 4. [Coordinación](#)
 5. [Nombrado y Localización](#)
 6. [Consistencia y Replicación](#)
 7. [Tolerancia a Fallos](#)
 8. [Seguridad](#)
-

1. Arquitectura o el problema de cómo diseñar el sistema

1.1 Organización del Sistema Distribuido

El sistema **File Search** está diseñado siguiendo una arquitectura de **microservicios con alta disponibilidad** desplegada sobre una infraestructura Docker Swarm. La arquitectura implementa un modelo **PRIMARY-BACKUP** con failover automático, organizada en tres capas principales:



PROF

1.2 Roles del Sistema

Rol	Componente	Descripción
Cliente (PRIMARY)	Streamlit App	Nodo principal de interfaz web con resolución DNS dinámica y reintentos
Cliente (BACKUP)	Streamlit App	Nodos de respaldo que pueden ser promovidos a PRIMARY
Servicio de Nombres (PRIMARY)	DNS Service	Coordinador principal del clúster, asigna roles a los servidores API
Servicio de Nombres (BACKUP)	DNS Service	Réplicas que sincronizan estado y pueden asumir el rol de PRIMARY

Rol	Componente	Descripción
Servidor de Aplicación (PRIMARY)	FastAPI + SQLite + Files	Nodo principal que atiende peticiones, contiene DB y archivos internos
Servidor de Aplicación (BACKUP)	FastAPI + SQLite + Files	Nodos de respaldo con DB y archivos sincronizados, pueden ser promovidos a PRIMARY

1.3 Distribución de Servicios en las Redes Docker

El sistema utiliza una red **overlay** (`file_search_net`) que permite la comunicación entre nodos del swarm. Los servicios se distribuyen estratégicamente entre dos nodos para garantizar que cada nodo tenga una combinación de PRIMARY y BACKUP:

Nodo 1 (Manager):

- Client 1 (PRIMARY Cliente)
- DNS Service 1 (PRIMARY DNS)
- Server 1 (PRIMARY API + SQLite + Files)
- Client 3 (BACKUP Cliente)
- DNS Service 3 (BACKUP DNS)
- Server 3 (BACKUP API + SQLite + Files)

Nodo 2 (Worker):

- Client 2 (BACKUP Cliente)
- DNS Service 2 (BACKUP DNS)
- Server 2 (BACKUP API + SQLite + Files)
- *(Réplicas adicionales de respaldo)*

Esta distribución garantiza que si un nodo falla completamente, el otro nodo tiene al menos un PRIMARY o puede promover un BACKUP a PRIMARY para cada servicio.

PROF

```
# Configuración de red en stack.yml
networks:
  file_search_net:
    driver: overlay
    attachable: true
```

1.4 Modelo PRIMARY-BACKUP

El sistema implementa un modelo de alta disponibilidad donde:

1. **Un solo PRIMARY activo:** Atiende todas las peticiones de escritura y lectura
2. **Múltiples BACKUPS pasivos:** Sincronizan datos del PRIMARY cada 10 segundos
3. **Failover automático:** Si el PRIMARY falla, un BACKUP es promovido automáticamente
4. **Coordinación via DNS:** El servicio DNS gestiona los roles y detecta fallos mediante heartbeats

2. Procesos o el problema de cuántos programas o servicios posee el sistema

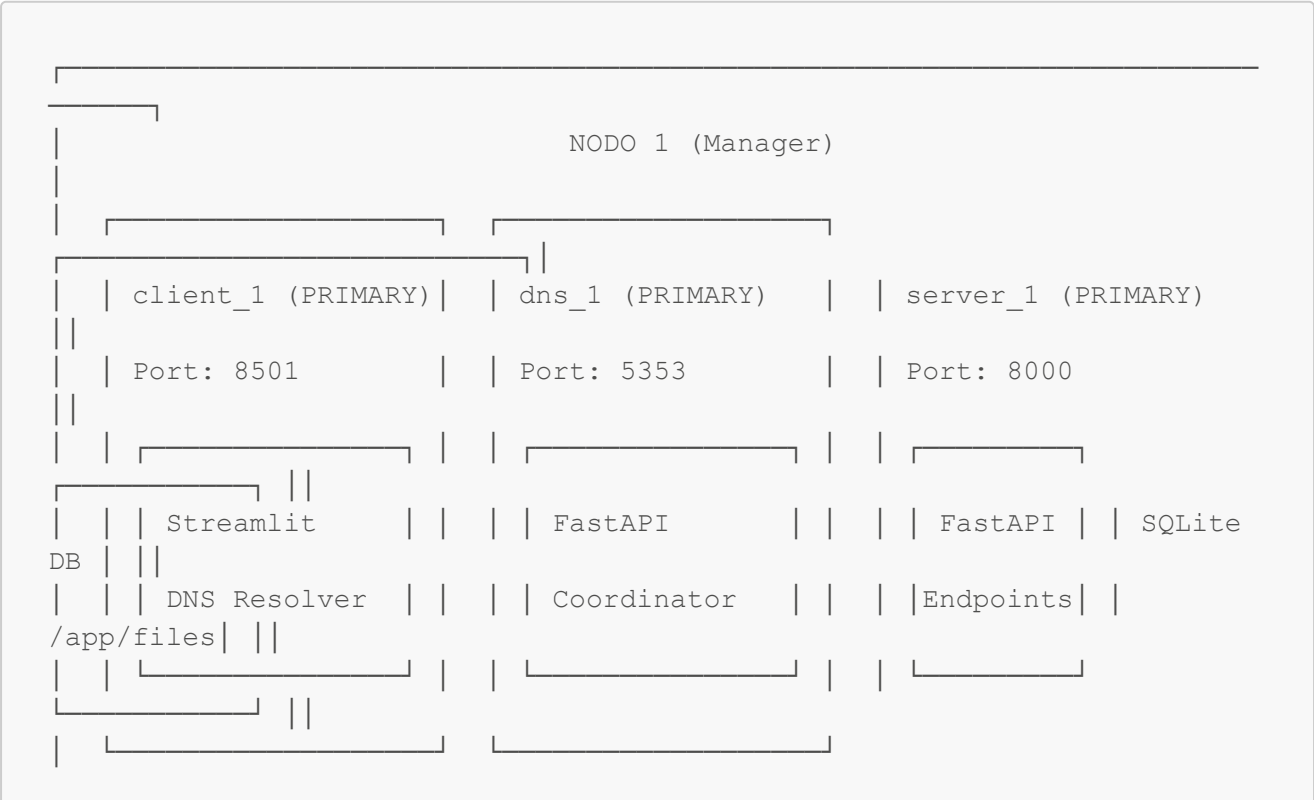
2.1 Tipos de Procesos dentro del Sistema

El sistema cuenta con **9 procesos principales** organizados en clústeres de alta disponibilidad, distribuidos en 3 tipos de servicios:

Proceso	Tipo	Tecnología	Puerto	Réplicas	Nodo
client_1	Cliente (PRIMARY)	Streamlit	8501	1	Nodo 1
client_2, client_3	Cliente (BACKUP)	Streamlit	8502, 8503	2	Nodo 1, Nodo 2
dns_1	Servicio DNS (PRIMARY)	FastAPI + Uvicorn	5353	1	Nodo 1
dns_2, dns_3	Servicio DNS (BACKUP)	FastAPI + Uvicorn	5354, 5355	2	Nodo 1, Nodo 2
server_1	Servidor API + DB + Files (PRIMARY)	FastAPI + Uvicorn + SQLite	8000	1	Nodo 1
server_2, server_3	Servidor API + DB + Files (BACKUP)	FastAPI + Uvicorn + SQLite	8001, 8002	2	Nodo 1, Nodo 2

2.2 Organización de los Procesos

Los procesos se organizan en **dos nodos físicos** para garantizar alta disponibilidad. Cada nodo contiene una mezcla de servicios PRIMARY y BACKUP:





Contenedores Docker: Un Proceso por Contenedor

Cada servicio se ejecuta en su **propio contenedor Docker aislado**. No hay múltiples servicios compartiendo el mismo contenedor:

PROF

Contenedor	Proceso Principal	Componentes Internos (mismo proceso)
client_1,client_2,client_3	streamlit run app.py	Streamlit App + DNS Resolver (módulo interno)
dns_1	uvicorn main:app	FastAPI + Coordinator + State Sync (corrutinas)
dns_2,dns_3	uvicorn main:app	FastAPI + State Sync (corrutinas)
server_1,server_2,server_3	uvicorn main:app	FastAPI + SQLite + NodeManager + SyncService (corrutinas)

Nota importante: Aunque cada contenedor ejecuta un solo proceso principal, los componentes como `NodeManager`, `SyncService` y `Coordinator` se ejecutan como **corrutinas asíncronas** dentro del mismo proceso Python (event loop), no como procesos o hilos separados.

Responsabilidades por Tipo de Contenedor

Tipo de Contenedor	Responsabilidades
client_N	Interfaz web, resolución DNS dinámica, reintentos automáticos
dns_N	Resolución de nombres, coordinación de roles PRIMARY/BACKUP, sincronización de estado entre DNS
server_N	API REST, almacenamiento de datos (SQLite + archivos), gestión de nodo, sincronización con PRIMARY

2.3 Patrón de Diseño con Respecto al Desempeño

El sistema emplea un **modelo asíncrono basado en event loop** para maximizar el rendimiento:

Patrón Async/Await (FastAPI + Uvicorn)

Todos los servidores (DNS y API) utilizan programación asíncrona para manejar múltiples conexiones concurrentes sin bloquear:

```
# Endpoints asíncronos en FastAPI
@app.post("/upload")
async def upload_file_endpoint(
    request: Request,
    file: UploadFile = File(...),
    folder: str = Form(None)
):
    # Operaciones async para manejo de archivos
    content = await file.read()
    ...
```

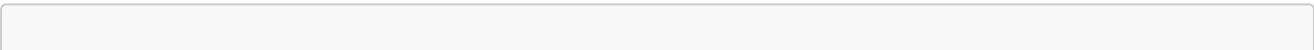
PROF

Tareas de Fondo con asyncio

Los servicios de sincronización y heartbeat se ejecutan como tareas asíncronas concurrentes:

```
@app.on_event("startup")
async def startup_event():
    # Tareas de fondo ejecutándose en paralelo
    asyncio.create_task(node_manager.heartbeat_loop()) # Cada 5s
    if node_manager.is_backup:
        asyncio.create_task(sync_service.sync_loop()) # Cada 10s
```

Caché con TTL para Reducir Latencia



```
# Cliente: Caché de resultados de búsqueda
@st.cache_data(ttl=10)
def fetch_files(query: str, *, limit: int, offset: int) -> List[Dict]:
    # Resultados cacheados por 10 segundos

# DNS Client: Caché de resoluciones
class DNSClient:
    def __init__(self, ...):
        self._cache: Dict[str, Dict[str, Any]] = {} # Caché local con
        TTL
```

Resumen del Patrón de Concurrency

Componente	Patrón	Tecnología	Beneficio
API Server	Async I/O	asyncio + uvicorn	Miles de conexiones concurrentes
DNS Service	Async I/O	asyncio + uvicorn	Baja latencia en resolución
Heartbeat Loop	Background Task	asyncio.create_task	No bloquea peticiones
Sync Service	Background Task	asyncio.create_task	Sincronización no bloqueante
Cliente	Caché TTL	st.cache_data	Reduce llamadas a API

3. Comunicación o el problema de cómo enviar información mediante la red

3.1 Tipo de Comunicación

El sistema utiliza **REST (Representational State Transfer)** como protocolo principal de comunicación, implementado sobre HTTP/1.1.

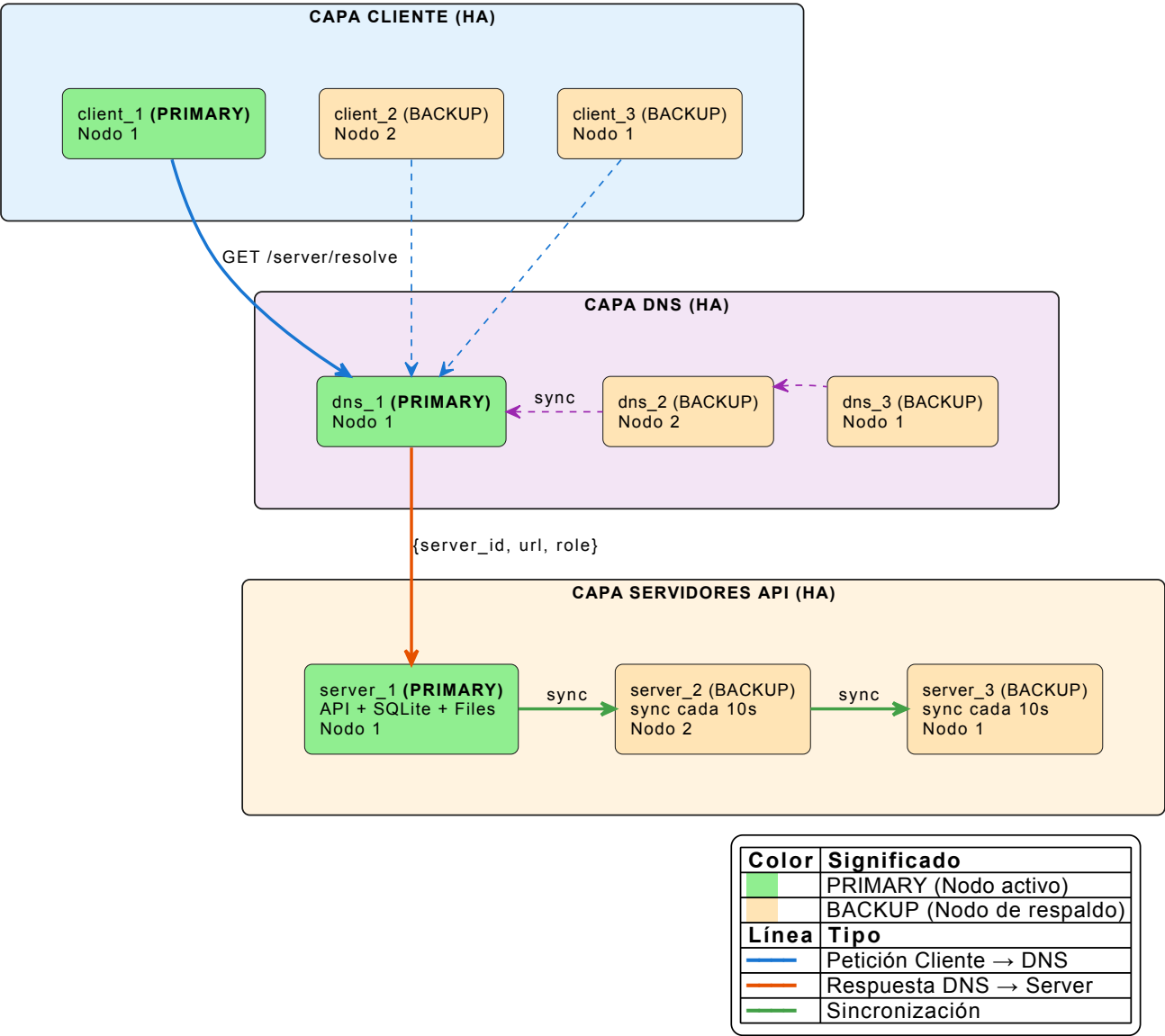
Características:

- Comunicación sin estado (stateless)
- Intercambio de datos en formato JSON
- Métodos HTTP estándar (GET, POST, DELETE)
- Endpoints internos para sincronización entre servidores

3.2 Comunicación Cliente-Servidor (con HA)

El sistema cuenta con **3 clientes**, **3 servicios DNS** y **3 servidores API**, todos con modelo PRIMARY-BACKUP distribuidos en 2 nodos físicos:

Comunicación Cliente-Servidor con Alta Disponibilidad
Modelo PRIMARY-BACKUP en 3 Capas



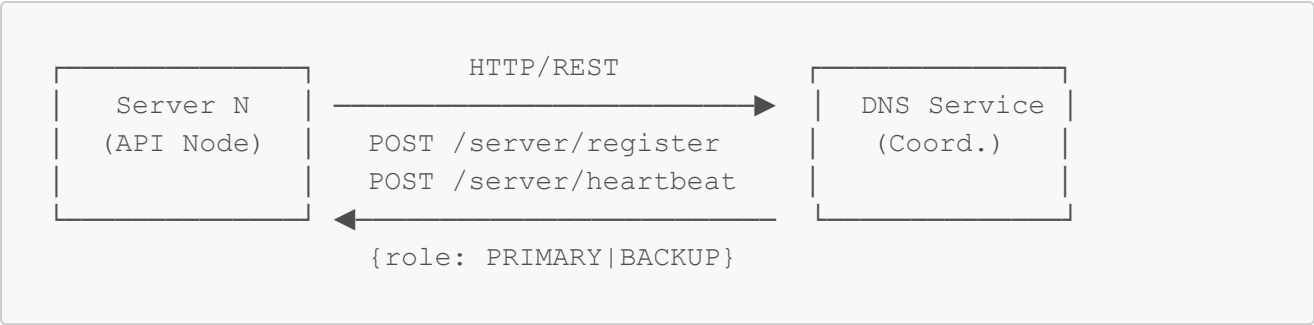
Endpoints principales:

Método	Ruta	Descripción
GET	/health	Verifica estado del servicio
GET	/search?query&limit&offset	Búsqueda paginada de archivos
GET	/files	Lista todos los archivos
GET	/files/{file_id}/download	Descarga un archivo
POST	/files	Registra/actualiza un archivo
POST	/upload	Sube un nuevo archivo
DELETE	/files/{file_id}	Elimina un archivo

Endpoints internos (sincronización entre servidores):

Método	Ruta	Descripción
GET	<code>/internal/db_snapshot</code>	Descarga snapshot de la base de datos
GET	<code>/internal/files</code>	Lista archivos con metadatos para sincronización
GET	<code>/internal/file/{path}</code>	Descarga un archivo específico para sincronización

3.3 Comunicación Servidor-DNS (Coordinación del Clúster)



Endpoints DNS para servidores API:

Método	Ruta	Descripción
POST	<code>/server/register</code>	Registra un servidor API en el clúster
POST	<code>/server/heartbeat</code>	Envía heartbeat y recibe rol actual
GET	<code>/server/resolve</code>	Obtiene URL del servidor PRIMARY
GET	<code>/server/list</code>	Lista todos los servidores y sus estados

3.4 Flujo de Comunicación con Reintentos

El cliente implementa un sistema de reintentos con re-resolución DNS:

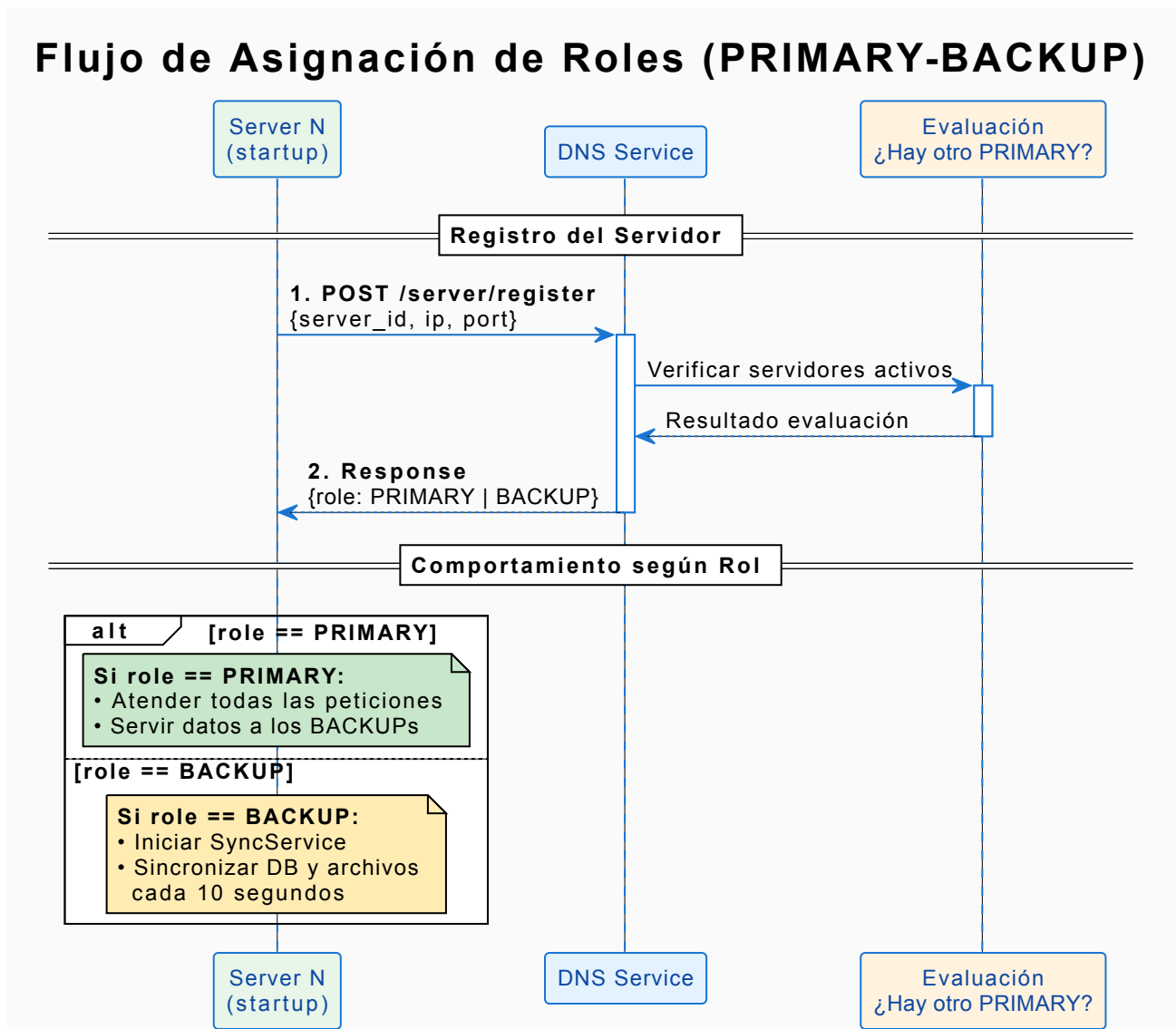
```
def make_request_with_retry(method: str, path: str, **kwargs) -> requests.Response:
    """
    Realiza una petición HTTP con reintentos y re-resolución DNS.
    """
    for attempt in range(1, MAX_RETRIES + 1): # MAX_RETRIES = 3
        try:
            url = api_url(path) # Resuelve via DNS
            response = requests.get(url, timeout=15, **kwargs)
            response.raise_for_status()
            return response
        except (requests.ConnectionError, requests.Timeout) as e:
            # Invalida cache y fuerza re-resolución en siguiente intento
            _invalidate_server_cache()
            if attempt < MAX_RETRIES:
                time.sleep(RETRY_DELAY) # RETRY_DELAY = 0.5s
    raise last_exception
```

4. Coordinación o el problema de poner todos los servicios de acuerdo

4.1 Coordinación de Roles (PRIMARY-BACKUP)

El sistema implementa un mecanismo de coordinación centralizada a través del servicio DNS:

Flujo de asignación de roles:



4.2 Detección de Fallos mediante Heartbeats

```
# En NodeManager (cada servidor API)
async def heartbeat_loop(self):
    while self._running:
        response = await client.post(
            f"{self._dns_url}/server/heartbeat",
            json={"server_id": self.server_id, "current_role":
self._role})
```

```

    )
    # El DNS puede cambiar nuestro rol en la respuesta
    new_role = data.get("role")
    if new_role != self._role:
        await self._handle_role_change(new_role, data)

    await asyncio.sleep(HEARTBEAT_INTERVAL) # 5 segundos

# En DNS Service (detección de servidores caídos)
async def check_api_servers():
    for server_id, info in api_servers.items():
        seconds_since_heartbeat = (now -
info["last_heartbeat"]).total_seconds()
        if seconds_since_heartbeat > API_SERVER_TIMEOUT: # 15 segundos
            # Servidor considerado caído
            if info["role"] == "PRIMARY":
                # Promover un BACKUP a PRIMARY
                await promote_backup_to_primary()

```

4.3 Sincronización de Acciones

1. **Inicialización del Servidor:** Al arrancar, el servidor espera al DNS y se registra:

```

@app.on_event("startup")
async def startup_event():
    init_db() # Inicializa base de datos local
    summary = sync() # Sincroniza archivos locales

    # Registrar con el clúster
    await node_manager.wait_for_dns() # Espera hasta 10 reintentos
    await node_manager.register() # Obtiene rol del DNS

    # Iniciar tareas de fondo
    asyncio.create_task(node_manager.heartbeat_loop())
    if node_manager.is_backup:
        asyncio.create_task(sync_service.sync_loop())

```

Nota: **Orden de Arranque de Servicios:** Debe asegurarse el orden de inicio de los servicios. Primero el sistema de DNS, luego los servidores y por último los clientes.

4.4 Acceso Exclusivo a Recursos - Condiciones de Carrera

En un sistema distribuido, múltiples procesos pueden intentar acceder o modificar los mismos recursos simultáneamente. Una **condición de carrera** ocurre cuando el resultado de una operación depende del orden impredecible en que se ejecutan las operaciones concurrentes, llevando a estados inconsistentes o errores.

Escenarios de Condiciones de Carrera en File Search

Escenario 1: Registro simultáneo de servidores

Imaginemos que `server_1` y `server_2` arrancan casi al mismo tiempo y ambos solicitan registrarse al DNS:

```
Tiempo T1: server_1 pregunta "¿Hay PRIMARY?" → DNS responde "No"
Tiempo T2: server_2 pregunta "¿Hay PRIMARY?" → DNS responde "No"
Tiempo T3: server_1 se registra como PRIMARY
Tiempo T4: server_2 se registra como PRIMARY ← ¡ERROR! Dos PRIMARY
```

Sin protección, ambos servidores podrían terminar siendo PRIMARY, violando la invariante del sistema de "un único PRIMARY activo".

Escenario 2: Actualización concurrente de archivos

Si dos peticiones intentan actualizar el mismo archivo simultáneamente:

```
Cliente A: Lee archivo X (versión 1)
Cliente B: Lee archivo X (versión 1)
Cliente A: Escribe archivo X con cambios de A → versión 2
Cliente B: Escribe archivo X con cambios de B → versión 2 ← ¡Cambios de A perdidos!
```

Escenario 3: Heartbeat y promoción simultáneos

```
DNS: Detecta que PRIMARY no responde, inicia promoción de BACKUP
PRIMARY: Se recupera y envía heartbeat justo en ese momento
Resultado: ¿Quién es el PRIMARY ahora?
```

PROF

Estrategias de Protección Implementadas

El sistema emplea tres mecanismos complementarios para evitar condiciones de carrera:

Mecanismo	Problema que Resuelve	Dónde se Aplica
Lock asíncrono	Acceso concurrente a estructuras en memoria	Registro de servidores en DNS
Operaciones atómicas (UPSERT)	Lecturas y escrituras no atómicas en DB	Inserción/actualización de archivos
Transacciones con rollback	Operaciones parcialmente completadas	Todas las operaciones de base de datos

Implementación de los Mecanismos

1. Lock asíncrono para operaciones en api_servers:

```
# En DNS Service
api_servers_lock = asyncio.Lock()

async def register_api_server(request: APIServerRegisterRequest):
    async with api_servers_lock:
        # Operaciones atómicas sobre api_servers
        if not any(s["role"] == "PRIMARY" for s in
api_servers.values()):
            role = "PRIMARY"
        else:
            role = "BACKUP"
        api_servers[request.server_id] = {...}
```

El lock garantiza que solo un "hilo" de ejecución (corrutina) puede estar dentro de la sección crítica a la vez. Mientras un servidor se está registrando, cualquier otro registro debe esperar.

2. Operaciones Atómicas con UPSERT:

El problema clásico de "leer-verificar-escribir" (check-then-act) se resuelve combinando la verificación y la escritura en una única operación atómica a nivel de base de datos:

```
INSERT INTO files (file_id, name, path, size, last_modified)
VALUES (?, ?, ?, ?, ?)
ON CONFLICT(file_id) DO UPDATE SET
    name = excluded.name,
    path = excluded.path,
    size = excluded.size,
    last_modified = excluded.last_modified
```

PROF

En lugar de hacer `SELECT` para verificar si existe y luego `INSERT` o `UPDATE`, la instrucción `ON CONFLICT ... DO UPDATE` realiza ambas operaciones en un solo paso atómico. La base de datos garantiza que no habrá interferencia entre operaciones concurrentes.

3. Context Manager para Transacciones:

Las transacciones aseguran que un conjunto de operaciones se ejecute "todo o nada". Si algo falla a mitad de camino, se deshacen todos los cambios (rollback), evitando estados intermedios inconsistentes:

```
@contextmanager
def get_conn():
    conn = sqlite3.connect(...)
    try:
        yield conn
        conn.commit()
    except Exception:
```

```

conn.rollback()
raise
finally:
    conn.close()

```

5. Nombrado y Localización o el problema de dónde se encuentra un recurso y cómo llegar al mismo

5.1 Identificación de Datos y Servicios

Identificación de Archivos:

Los archivos se identifican mediante un hash SHA-1 de su ruta relativa:

```

def _compute_file_id(relative_path: str) -> str:
    digest = hashlib.sha1(relative_path.encode("utf-8"))
    return digest.hexdigest()

```

Identificación de Servicios:

Servicio	Nombre DNS	Puerto
API Server	server	8000
DNS Service	dns_service	5353
Client	client	8501

5.2 Ubicación de Datos y Servicios

Ubicación de Datos (internos a cada contenedor server_N):

- **Metadatos:** Base de datos SQLite en `/app/server/db/doc_search.db`
- **Archivos físicos:** Directorio interno `/app/files` (no volumen persistente)
- **Logs:** Volumen compartido `/app/logs` (único volumen externo)

Nota: Los servicios están distribuidos en 2 nodos físicos. Esta distribución garantiza que si un nodo falla, el otro tiene servicios que pueden ser promovidos a PRIMARY.

5.3 Localización de Datos y Servicios

El sistema implementa un **servicio DNS personalizado** para la localización de servicios:





Flujo de Resolución:

1. El cliente solicita la IP del servicio `server` al DNS Service
2. El DNS Service consulta el DNS interno de Docker
3. El DNS Service devuelve la IP resuelta con un TTL
4. El cliente cachea la respuesta y la utiliza para comunicarse

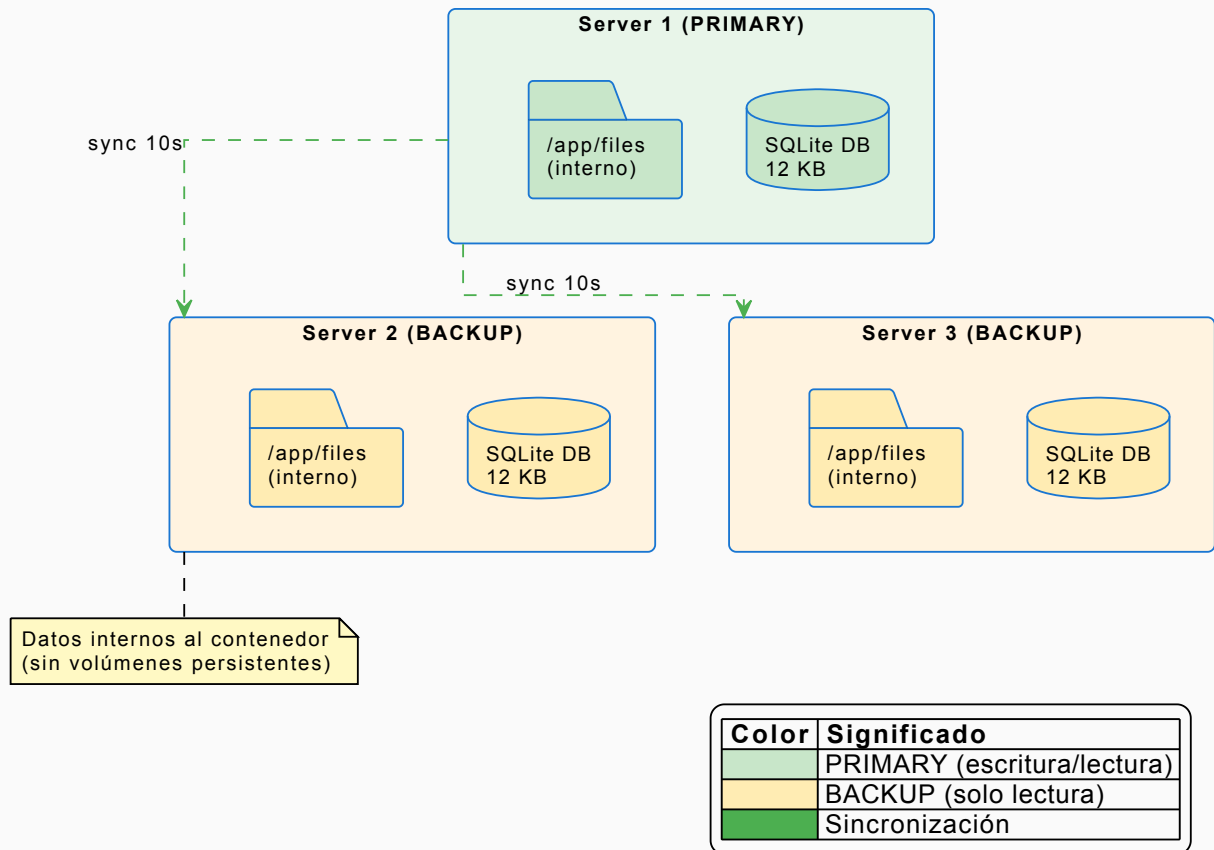
```
class DNSClient:
    def resolve(self, hostname: str) -> str:
        # 1. Revisa caché local (TTL)
        # 2. Consulta al DNS Service vía API
        # 3. Fallback a resolución nativa
```

6. Consistencia y Replicación o el problema de solucionar los problemas que surgen a partir de tener varias copias de un mismo dato en el sistema

6.1 Distribución de los Datos

El sistema implementa replicación completa con un modelo PRIMARY-BACKUP:

Arquitectura de Datos - Replicación PRIMARY-BACKUP



Características del almacenamiento:

- **Datos internos al contenedor:** No se usan volúmenes persistentes para datos
- **Solo logs en volúmenes:** Los logs se comparten via volumen para debugging
- **Copia inicial al arrancar:** Los archivos se copian desde un volumen temporal de solo lectura al iniciar el contenedor

6.2 Replicación - Cantidad de Réplicas

PROF

Configuración actual:

Componente	Réplicas	Rol	Distribución
Cliente	3	1 PRIMARY + 2 BACKUP	Nodo 1: client_1 (P), client_3 (B); Nodo 2: client_2 (B)
DNS Service	3	1 PRIMARY + 2 BACKUP	Nodo 1: dns_1 (P), dns_3 (B); Nodo 2: dns_2 (B)
API Server	3	1 PRIMARY + 2 BACKUP	Nodo 1: server_1 (P), server_3 (B); Nodo 2: server_2 (B)

6.3 Mecanismo de Sincronización

SyncService implementa sincronización cada 10 segundos:


```

class SyncService:
    async def sync_database(self) -> bool:
        """Descarga snapshot de DB usando sqlite3.backup() para
        consistencia."""
        async with httpx.AsyncClient(timeout=30) as client:
            response = await client.get(f"
{primary_url}/internal/db_snapshot")

            # Guardar en archivo temporal
            with tempfile.NamedTemporaryFile(delete=False) as tmp:
                tmp.write(response.content)

            # Verificar integridad
            conn = sqlite3.connect(tmp_path)
            cursor.execute("PRAGMA integrity_check")

            # Reemplazar base de datos local
            shutil.move(tmp_path, str(DB_PATH))

    async def sync_files(self) -> Dict[str, int]:
        """Sincroniza archivos nuevos/modificados."""
        # Obtener lista de archivos del PRIMARY
        response = await client.get(f"{primary_url}/internal/files")
        remote_files = response.json()["files"]

        for file in remote_files:
            if not local_exists or local_size != remote_size:
                # Descargar archivo
                await download_file(file["relative_path"])

```

6.4 Consistencia tras Actualización

El sistema garantiza consistencia eventual con los siguientes mecanismos:

1. **Sincronización al inicio:** Cada servidor sincroniza archivos locales al arrancar desde el volumen temporal:

```

# entrypoint.sh
cp -r "$SOURCE_FILES_DIR"/* "$INTERNAL_FILES_DIR"/

```

2. **Identificadores estables:** Los file_id basados en hash garantizan identificación consistente:

```

def _compute_file_id(relative_path: str) -> str:
    return hashlib.sha1(relative_path.encode("utf-8")).hexdigest()

```

3. **Operaciones idempotentes:** UPSERT permite reintentos sin duplicación.

4. **Verificación de integridad:** Antes de reemplazar la DB, se verifica con `PRAGMA integrity_check.`

7. Tolerancia a Fallos o el problema de para qué pasar tanto trabajo distribuyendo datos y servicios si al fallar una componente del sistema todo se viene abajo

7.1 Nivel de Tolerancia a Fallos Alcanzado

El sistema implementa **Nivel 2 de tolerancia a fallos:**

Escenario	Comportamiento	Tiempo de Recuperación
Caída del PRIMARY	BACKUP promovido automáticamente	~15 segundos (timeout)
Caída de 1 BACKUP	Sistema continúa operando	Sin impacto
Caída de PRIMARY + 1 BACKUP	Último BACKUP se convierte en PRIMARY	~15 segundos
Caída de 2 BACKUPS	PRIMARY continúa operando solo	Sin impacto
Reinicio de servidor caído	Re-registra como BACKUP y sincroniza	~10 segundos

7.2 Mecanismos de Detección de Fallos

1. Heartbeats con timeout:

```
# Servidor envía heartbeat cada 5 segundos
HEARTBEAT_INTERVAL = 5

# DNS considera servidor caído tras 15 segundos sin heartbeat
API_SERVER_TIMEOUT = 15

# Verificación en DNS
async def check_api_servers():
    async with api_servers_lock:
        for server_id, info in api_servers.items():
            seconds = (now - info["last_heartbeat"]).total_seconds()
            info["alive"] = seconds <= API_SERVER_TIMEOUT

# Si PRIMARY está caído, promover un BACKUP
primary = get_primary()
if primary and not primary["alive"]:
    await promote_backup_to_primary()
```

2. Promoción automática de BACKUP a PRIMARY:

```

async def promote_backup_to_primary():
    """Promueve el primer BACKUP disponible a PRIMARY."""
    for server_id, info in api_servers.items():
        if info["role"] == "BACKUP" and info["alive"]:
            info["role"] = "PRIMARY"
            logger.info(f"[DNS] {server_id} promovido a PRIMARY")

            # El servidor recibirá su nuevo rol en el próximo heartbeat
            break

```

7.3 Respuesta a Errores

1. Cliente con reintentos y failover:

```

def make_request_with_retry(method: str, path: str, **kwargs):
    for attempt in range(1, MAX_RETRIES + 1): # MAX_RETRIES = 3
        try:
            url = api_url(path) # Resuelve via DNS
            response = requests.get(url, timeout=15)
            return response
        except (requests.ConnectionError, requests.Timeout):
            # Invalida cache para forzar re-resolución DNS
            _invalidate_server_cache()
            time.sleep(RETRY_DELAY) # 0.5 segundos

```

2. Fallback en resolución DNS:

```

def get_api_base_url() -> str:
    # Intenta resolver via DNS
    server_url = _resolve_server_from_dns()
    if server_url:
        return server_url

    # Fallback: Usar variable de entorno
    return os.getenv("API_BASE_URL", "http://localhost:8000")

```

7.4 Escenarios de Failover Probados

Escenario 1: Caída del PRIMARY

Estado inicial: server_1 (PRIMARY), server_2 (BACKUP), server_3 (BACKUP)
 Acción: docker compose stop server_1
 Resultado: Tras 15s, server_2 o server_3 promovido a PRIMARY
 Datos: Disponibles (sincronizados previamente)

Escenario 2: Caída de PRIMARY + 1 BACKUP

```
Estado: server_1 (PRIMARY), server_2 (BACKUP), server_3 (BACKUP)
Acción: docker compose stop server_1 server_2
Resultado: server_3 promovido a PRIMARY
Datos: Disponibles
```

Escenario 3: Reincorporación de servidor

```
Estado: server_2 (PRIMARY)
Acción: docker compose start server_1
Resultado: server_1 re-registra como BACKUP, inicia sincronización
```

8. Seguridad o el problema de qué tan vulnerable es su diseño

8.1 Seguridad con Respecto a la Comunicación

Estado actual:

- Comunicación HTTP sin cifrado dentro del clúster Docker
- Red overlay aislada del exterior
- Puertos expuestos controlados por el firewall de Docker

Mejoras planificadas:

- Implementación de HTTPS con certificados TLS
- Mutual TLS (mTLS) para comunicación entre servicios
- API Gateway con rate limiting

8.2 Seguridad con Respecto al Diseño

Medidas implementadas:

1. **Aislamiento de contenedores:** Cada servicio corre en su propio contenedor con recursos limitados.
2. **Red overlay privada:** Los servicios se comunican en una red interna no accesible externamente.
3. **Validación de entrada con Pydantic:**

```
class FileIn(BaseModel):
    file_id: str
    name: str
    path: str
    size: int
    last_modified: datetime
```

4. Manejo seguro de archivos:

```
# Control de directorios de destino
target_dir = _DEFAULT_ROOT
if folder:
    target_dir = target_dir / folder
    os.makedirs(target_dir, exist_ok=True)
```

Vulnerabilidades conocidas y mitigaciones planificadas:

Vulnerabilidad	Riesgo	Mitigación Planificada
SQL Injection	Bajo (usa parametrización)	Auditoría de queries
Path Traversal	Medio	Validación de rutas
DoS	Medio	Rate limiting, quotas

8.3 Autorización y Autenticación

Estado actual:

- Sin autenticación implementada
- Acceso abierto a todos los endpoints

Mejoras planificadas para cumplir requisitos adicionales:

1. Autenticación JWT:

- Tokens de acceso con tiempo de expiración
- Refresh tokens para renovación

2. Roles y permisos:

- Usuario: lectura y descarga
- Administrador: upload y eliminación

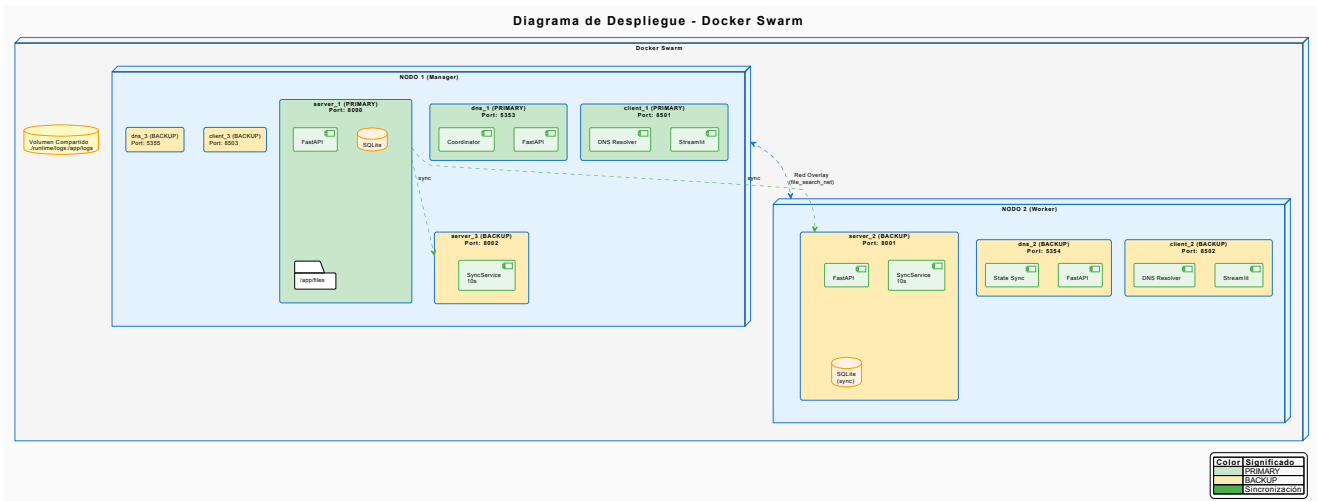
3. Auditoría de accesos:

- Logging estructurado ya implementado
- Access logs separados de application logs

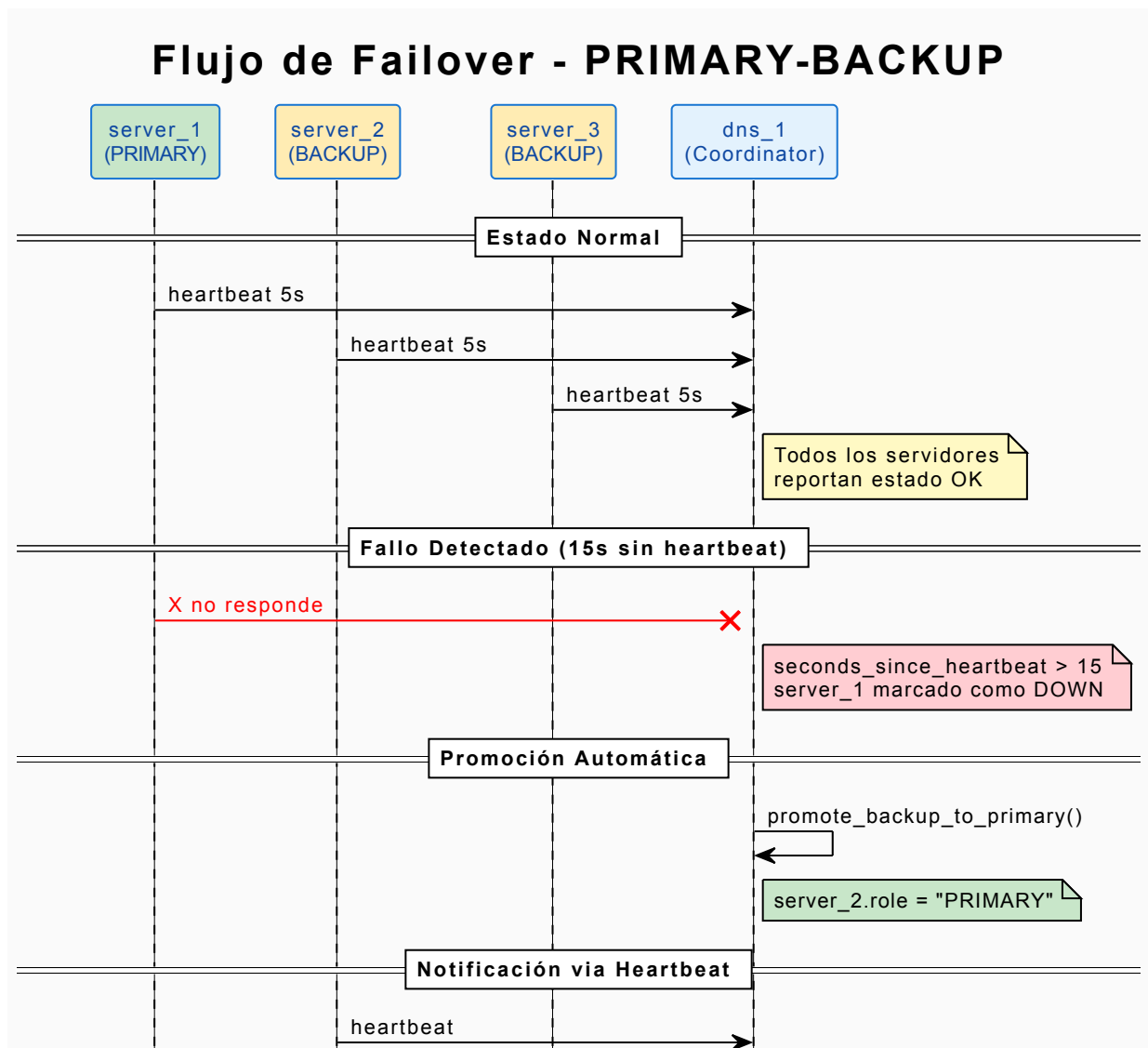
```
access_logger = logging.getLogger("app.access")
access_logger.info(
    "%s %s %s -> descarga '%s'",
    client_host,
    request.method,
    request.url.path,
```

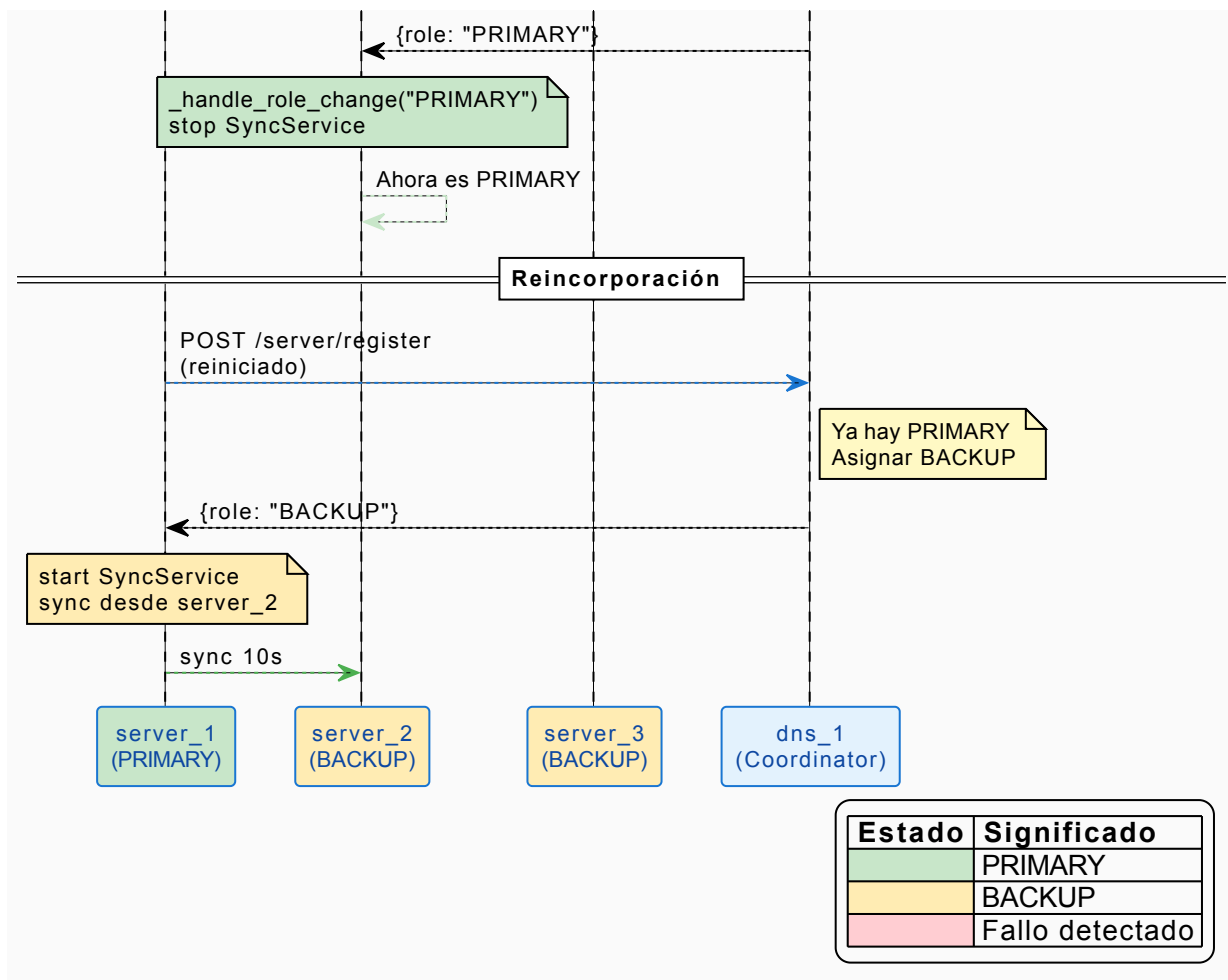
```
target.filename,  
)
```

Diagrama de Despliegue



Flujo de Failover





Conclusiones

El sistema **File Search** implementa una arquitectura de microservicios con **alta disponibilidad** sobre Docker Swarm que alcanza:

Logros Implementados

1. Tolerancia a Fallos Nivel 2:

- El sistema puede sobrevivir a la caída de hasta 2 servicios de cada tipo
- Failover automático en ~15 segundos
- Re-sincronización automática al reincorporar nodos
- Distribución estratégica en 2 nodos con mezcla de PRIMARY/BACKUP

2. Replicación Completa en 3 Capas:

- 3 réplicas del cliente (1 PRIMARY + 2 BACKUP)
- 3 réplicas del servicio DNS (1 PRIMARY + 2 BACKUP)
- 3 réplicas del servidor API (1 PRIMARY + 2 BACKUP)
- Sincronización de base de datos y archivos cada 10 segundos

3. Coordinación Centralizada:

- DNS Service como coordinador del clúster

- Asignación dinámica de roles (PRIMARY/BACKUP)
- Detección de fallos mediante heartbeats (5s intervalo, 15s timeout)

4. Almacenamiento Interno (Sin Volúmenes Persistentes):

- Datos internos a cada contenedor server_N (no volúmenes persistentes)
- Solo logs en volúmenes compartidos para debugging
- Copia inicial desde volumen temporal de solo lectura

5. Cliente con Alta Disponibilidad:

- 3 instancias con modelo PRIMARY-BACKUP
- Resolución DNS dinámica del servidor PRIMARY
- Reintentos automáticos con re-resolución
- Failover transparente al usuario

Métricas del Sistema

Métrica	Valor
Intervalo de heartbeat	5 segundos
Timeout de servidor	15 segundos
Intervalo de sincronización	10 segundos
Reintentos del cliente	3
Tiempo de failover	~15 segundos

Componentes Implementados

Archivo	Descripción
<code>app/server/services/node_manager.py</code>	Gestión del nodo en el clúster
<code>app/server/services/sync_service.py</code>	Sincronización de datos para BACKUPS
<code>app/dns_service/main.py</code>	Coordinador del clúster con gestión de roles
<code>app/client/app.py</code>	Cliente con resolución DNS y reintentos
<code>app/common/resolver.py</code>	Módulo DNS Resolver compartido por clientes
<code>docker-compose.yml</code>	Configuración de 9 servicios (3 client + 3 DNS + 3 server)
<code>stack.yml</code>	Configuración para Docker Swarm con 2 nodos